

1 · FUNÇÕES (envolvem o valor)		
<code>int("123") → 123</code>	texto → inteiro; mata zero à esq. (<code>int("02") → 2</code>)	
<code>float("12.5") → 12.5</code>	texto → decimal (ponto, NUNCA vírgula)	
<code>str(42) → "42"</code> <code>input("msg")</code> <code>print(x, y)</code>	número → texto lê do usuário; retorna SEMPRE str mostra; vírgula põe espaço entre args	
<code>len(x)</code>	tamanho: lista, string, df (nº de linhas)	
<code>round(x, 2)</code>	arredonda em 2 casas	
<code>sum(lista) / max / min</code>	soma / maior / menor de uma lista	
<code>range(5)</code>	0,1,2,3,4 (pra <code>for i in range(n)</code>)	
<code>type(x)</code>	debug: mostra o tipo da variável	
△ Função RETORNA , não altera: sempre <code>x = int(x)</code> , nunca <code>int(x)</code> solto.		

2 · MÉTODOS DE STRING (depois)		
<code>s.split("/")</code>	cuta → LISTA de str (converter números!)	
<code>s.upper() / s.lower()</code> <code>s.strip()</code> <code>s.isdigit()</code>	MAIÚSCULA / minúscula (atribuir!) tira espaços das pontas True só se TUDO é dígito ("12.3", "-5", "12" → False)	
<code>s.replace("a", "b")</code> <code>s[0:4]</code>	troca todas as ocorrências fatia: posições 0,1,2,3 (fim NÃO entra)	
<code>s[6:8]+"/"+s[4:6]</code> <code>"x" in s / in lista</code>	remontar data fatiada contém?	
<code>a, b, c = s.split("/")</code>	desempacota: 1 nome por pedaço (nº igual!)	

3 · F-STRING (montar texto)		
<code>f"01 {nome}"</code>	<code>f</code> liga o modo; <code>{}</code> injeta variável	
<code>f"{x:.2f}"</code>	2 casas decimais (mata 3.59999...)	
<code>f"{x:.1f}"</code>	1 casa decimal	

△ UMA string contínua. Sem `+`, sem `{}` fora do `f"`. Escreve a frase literal primeiro, depois põe as chaves nas variáveis. Caractere especial do enunciado (-): COPIA.

3b · LAÇOS: FOR vs WHILE		
<code>for t in range(3):</code> <code>for item in lista:</code> <code>while saldo >= 4:</code> <code>saldo -= 4</code>	# nº de voltas CONHECIDO antes # visita cada item da coleção # repete ENQUANTO a condição valer # algo muda a cada volta	

△ Ambos rodam; o MAIS apropriado (justifical): quantidade fixa antes do laço = `for` - "enquanto tiver/der/valer" = `while`. "R\$40 dá 10 tentativas" é isca: `saldo` é variável, a regra de parada é condição, não contagem.

4 · CRIAR / LER DATAFRAME		
<code>import pandas as pd</code> <code>df = pd.read_excel("a.xlsx")</code> <code>pd.read_excel("a.xlsx", index_col=0)</code> <code>pd.read_csv("a.csv")</code>	sempre primeiro planilha Excel 1ª coluna vira índice CSV padrão (vírgula separa, ponto decimal)	
<code>pd.read_csv("a.csv", sep=";", decimal=",")</code>	CSV BRASILEIRO : sem isso nº vira texto	
<code>df = pd.DataFrame({ "time": ["Palmeiras", "Flamengo"], "pontos": [68, 68]})</code>	# do zero: dict # chave=coluna, lista=valores	

△ Teste pós-leitura: `df.dtypes` . Coluna de valor tem que ser `float64/int64`; se `object` = virou texto, faltou `decimal=","`. Arquivos da prova ficam na MESMA pasta do notebook: caminho = SÓ o nome do arquivo.

5 · EXPLORAR		
<code>df.head()</code> / <code>df.tail()</code>	5 primeiras / 5 últimas linhas (DF INTEIRO, todas col.)	
<code>df.shape</code> <code>df.columns / list(df.columns)</code>	(linhas, colunas), sem parênteses nomes EXATOS das colunas (acentos conta!)	
<code>df.dtypes</code> <code>df.info()</code> / <code>df.describe()</code>	tipo de cada coluna resumo geral / stats completos	

6 · SELECIONAR / FILTRAR / ALTERAR		
<code>df["col"]</code> <code>df["col"] == "SP"</code>	uma coluna (Series) MÁSCARA: True/False por linha (!= >= <...)	
<code>df[df["col"] == "SP"]</code> <code>df.loc[mask, "col"] = v</code>	só as linhas True (tabela filtrada) REGRA DE OURO p/ trocar valor filtrado	
<code>df.loc[mask, "col"] += 0.04</code> <code>df["nova"] = df["a"] * df["b"]</code> <code>linha = df[df["id"]==x].iloc[0]</code>	somar só nos filtrados criar coluna (vetorizado, sem laço) UMA linha como ficha → <code>linha["col"]</code>	
<code>x in df["col"].values</code> <code>df["col"].isin(["A","B"])</code> <code>df["col"].astype(str)</code>	existe? (SEM .values checa o índice!) máscara: está na lista? converte coluna (p/ comparar nº com texto)	
<code>df["col"].str[0:3]</code> <code>df["col"].str.upper()</code>	fatia cada linha (versão coluna do s[0:3]) .str dá os métodos de string na coluna	

△ NUNCA `df[df[...]==x]["col"] = v` (cópia, não altera).
FutureWarning/Chained na prova: IGNORAR (enunciado manda).

7 · AGREGAR / CONTAR		
<code>df["col"].sum()</code> . <code>mean()</code> . <code>min()</code> . <code>max()</code> <code>df["col"].agg(["min","mean","max"])</code> . <code>round(2)</code>	soma, média, min, máx vários stats em UMA operação	
<code>df["col"].value_counts()</code>	contagem POR categoria (tabela pronta)	
<code>df["col"].nunique()</code> / <code>.unique()</code>	QUANTAS distintas / QUAIS são	
<code>mask.sum()</code>	conta True (True=1); .count() = não-nulos, NÃO use	
<code>serie.sort_values()</code>	crescente; ascending=False = decrescente	
<code>serie.head(5)</code>	top 5 do ranking	

8 · ANÁLISE (Excel → pandas)		
<code>df.groupby("g")["v"].mean()</code>	média de v POR grupo (+ .sort_values(ascending=False) = ranking)	
<code>df.groupby("g")["v"].agg(["mean","max"])</code> <code>pd.merge(a, b, on=["k1","k2"])</code>	vários stats por grupo PROCv: traz colunas de b casando pelas chaves. Chave faltando MULTIPLICA linhas (teste: shape!)	
<code>df.pivot_table(index="Estado", columns="Produto", values="Total", aggfunc="mean").reset_index()</code>	Tabela Dinâmica: index=Linhas, columns=Colunas, values=Valores, aggfunc=Resumir por. .reset_index() devolve coluna comum; df.columns.name=None limpa rótulo	
<code>(df["m"] < 0).replace({True:"prejuízo", False:"lucro"})</code>	classificar sem if	
<code>np.where(cond, "sim", "não")</code>	idem (exige <code>import numpy as np</code>)	

△ TRIAGEM: dataset grande + média/ranking/cruzar = pandas SEM for. Lista de strings item a item = `for` + `split`. Escreveu `for` em análise? Pergunta: "qual ferramenta pandas faz isso?"

9 · ESQUELETOS		
# VALIDAÇÃO (valida ANTES de converter!) <code>d = input("Dist: ")</code> <code>uf = input("UF: ").upper()</code> <code>if not d.isdigit() or uf not in ["D","N"]:</code> <code>print("ERRO DE ENTRADA")</code> <code>else:</code> <code>d = int(d)</code> <code>if d <= 2000:</code> # "até X" INCLUI X <code>taxa = 5.0</code> <code>elif d <= 8000:</code> # <code>elif</code> = faixa seguinte <code>taxa = d/1000 * 1.2</code> # 1.2 PONTO! <code>else:</code> <code>taxa = 25.0</code> <code>if uf == "NM":</code> <code>taxa *= 2</code> # dobro vale p/ fixo tb <code>print(f"R\$ {taxa:.2f}")</code>		

# ACUMULADOR (Q5: lista de strings) <code>cont = 0; total = 0</code> # zera FORA <code>for item in lista:</code> # item = 1 string <code>partes = item.split(" ")</code> # da FRUTA, não da cesta <code>valor = float(partes[2])</code> # converte E atribui <code>cont += 1</code> <code>total += valor</code> <code>print(f"... {partes[0]} ...")</code> # 1 por item <code>print(f'{cont} x - Total: R\$ {total:.2f}')</code> # FORA (desindenta =		
---	--	--

# DATA C/ CAMPO OPCIONAL (split em estágios) <code>meses = ['janeiro','fevereiro','março','abril', 'maio','junho','julho','agosto','setembro', 'outubro','novembro','dezembro']</code> <code>for reg in datas:</code> <code>partes = reg.split(" ")</code> # grosso 1º <code>dia, mes, ano = partes[0].split("/")</code> <code>txt = f"{int(dia)} de {meses[int(mes)-1]} de {ano}"</code> <code>if len(partes) == 2:</code> # guarda! <code>h, m = partes[1].split(":")</code> <code>h = int(h); m = int(m)</code> <code>txt += " à 1 hora" if h==1 else f" às {h} horas"</code> <code>if m == 1:</code> <code>txt += " e 1 minuto"</code> <code>elif m > 1:</code> <code>txt += f" e {m} minutos"</code> <code>print(txt)</code> # m==0: omite		
--	--	--

# BUSCA POR ID + ÁRVORE (ordem = prioridade) <code>pid = int(input("id: "))</code> # int: coluna é nº! <code>if pid not in df["id"].values:</code> <code>print("Paciente não encontrado")</code> <code>else:</code> <code>linha = df[df["id"]==pid].iloc[0]</code> <code>imc = linha["peso"]/linha["altura"]**2</code> <code>if linha["sist"] >= 140 or linha["diast"] >= 90:</code> <code>r = "consulta imediata"</code> # regra 1 GANHA <code>elif imc >= 30 and linha["fum"]=="sim":</code> <code>r = "risco alto"</code> <code>elif imc >= 30 or linha["idade"] >= 60:</code> <code>r = "risco moderado"</code> # OU do enunciado = or <code>r = "risco baixo"</code> <code>print(f"Paciente {pid} (IMC: {imc:.1f}) - Risco: {r})")</code>		
---	--	--

O ERRO-MESTRE (5x na mesma noite)
Conversão sem atribuir / input é str. <code>int(x)</code> e <code>x.upper()</code> soltos NÃO mudam nada; <code>"1" == 1</code> é False; <code>"12"~1</code> e <code>"5"<18</code> são TypeError; <code>meses[mes-1]</code> com <code>mes</code> str quebra. REGRA: depois de TODO input ou split, pergunta "converto pra quê?" e faz <code>x = int(x)</code> na MESMA linha. Função retorna, variável só muda com <code>=</code>.
CONDIÇÕES (4 erros)
<code>is not</code> não compara valor. String se compara com <code>==</code> / <code>!=</code>. O warning do Python te DIZ o conserto: lê. <code>x != "D" or "N"</code> é sempre True. "N" sozinho é truthy. Cada lado do or/and precisa ser comparação COMPLETA. Idioma: <code>x not in ["D","N"]</code>. Com <code>!=</code> o conector é <code>and</code>. Conectivo do enunciado traduzido errado. "OU" = <code>or</code>, "E" = <code>and</code>, literal. E a ORDEM das regras é a prioridade do elif: regra 1 ganha da 2. Borda: "até X" INCLUI X. É <code><=</code>, não <code><</code>. Com <code><</code>, a entrada exata (2000, 8000, 15000) cai na faixa errada. Rubrica sem parcial: 3 bordas erradas = 1,5pt = zero.
NÚMEROS (2)
1,2 é TUPLA, não um-vírgula-dois. Decimal é PONTO: <code>1.2</code>. Se imprimir <code>(3.0, 2)</code>, é vírgula perdida em conta. 3.5999999999999996 NÃO é bug teu. Float binário não representa 1.2 exato. Não "conserta" a conta: formata com <code>{x:.2f}</code> e segue.
TABELA x LINHA x ITEM (3x o mesmo crash)
ValueError: truth value of a Series is ambiguous = você pôs a COLUNA inteira num if. · Depois de <code>linha = df[...].iloc[0]</code>: TUDO do paciente vem de <code>linha["col"]</code>, nunca <code>df["col"]</code>. · Dentro de <code>for i in coluna</code>: o valor da vez é <code>i</code>, compara com <code>i</code>. <code>df = todo mundo · linha = um registro · i = o item da vez</code> <code>in sem .values</code> cheça o ÍNDICE. Existência de valor: <code>x in df["col"].values</code>, com tipo batendo (int com int). <code>.count()</code> em máscara conta não-nulos (tudo), não os True. Contar True = <code>mask.sum()</code>.
LAÇOS E STRINGS (5)
<code>Split na CESTA: lista.split()</code> não existe. O for entrega 1 item por volta; split é no item. Dois separadores num split só não rola (" / " " vira ' / '). Split em ESTÁGIOS: espaço primeiro (separa mundos), barra depois (abre a data). Campo opcional fora da guarda = IndexError. <code>partes[1]</code> só existe se <code>len(partes)==2</code>; acessa SÓ dentro do if. (Gêmeo do "Paciente não encontrado"). <code>return</code> não fecha laço. Bloco fecha DESIDENTANDO (volta à margem). <code>return</code> é só de função (def). Misturar + com {} fora de f-string. Escolhe UM método: f-string inteira. Justaposição ("a" x) não existe; literais adjacentes ("a" "b") colam em silêncio.
PANDAS / MÉTODO (4)
Laço manual com contadores onde era <code>value_counts()</code>. Reflexo Excel de fazer na mão. Análise = 1 linha vetorizada (triagem da frente, seção 8). Método inventado (<code>.pos()</code>) e fatia em parênteses. Fatia (0:3) SÓ em colchetes. Não lembra o método? Olha a seção 6/7 da frente, não inventa. Literal errado: "Potes" ≠ "Pote". String é EXATA. Os valores reais saem de <code>df["col"].unique()</code> ou do head(). Trabalho duplicado nos 2 ramos do if/else. O que é comum vem ANTES do if; o if só guarda a divergência.

LEITURA DE QUESTÃO (3)

`head()` mostra o DF INTEIRO (5 linhas, TODAS as colunas), não "a coluna". Precisão custa 0,33/bloco.

Multiplicador: $\ast 0.85$ = REDUZ 15% (preço vira 85%). $\ast 0.2$ = vira 20% = reduz 80%. Lê o número antes de escrever a redução.

Responder Q2 em frase corrida única. Rubrica é por bloco: responde em 3 linhas, 1 por bloco (carrega / opera / mostra).

ENSAIO DA MANHÃ (5 furos novos, 10.06)

Questão em branco (Q1, valia 1,0 e estava na folha). Varredura final OBRIGATÓRIA: rolar o notebook inteiro; nenhuma célula de resposta vazia; Q0 preenchida.

Resposta em texto rodou como código → `SyntaxError`. Em célula de código, TODA linha de prosa começa com `#`.

Direção do ajuste: reajuste/aumento = $+=$ · desconto/redução = $--$. O módulo da diferença pode coincidir e esconder o erro: confere o `head()` depois da troca.

"Entre A (inclusive) e B (inclusive)" = $x \geq A$ and $x \leq B$. Invertido ($\leq A$ and $\geq B$) nada satisfaz. Typo em variável só explode no ramo não testado: rodar UMA entrada de CADA categoria/estado.

Preço via input = `float()`, nunca `int()`. E se o enunciado pede 3 valores na saída (original, final, diferença), imprime os 3: faltar 1 = rubrica inteira zero.

⚡ PROTOCOLO DE PROVA

0. Q0 primeiro	nome, turma AE_, código; não apagar os #
1. Triagem 60s	classifica as 5 questões: conceito / ler pandas / if+validação / DataFrame / parse em laço / análise
2. Ordem	Q1→Q2 (rápidas) → Q5 (3pts) → Q3 → Q4. Travou 10min: pula e volta
3. Antes do código	<code>df.columns + df.head()</code> : nome EXATO das colunas e valores reais
4. Consertos em lote	achou 3 problemas? conserta os 3, DEPOIS roda. Não 1 por vez
5. Teste de fechamento	exemplo do enunciado + borda exata de cada faixa + caso de ERRO + os 2 períodos/categorias. SÓ aí "tá pronto"
6. Warnings	<code>SyntaxWarning</code> te DIZ o conserto; <code>FutureWarning</code> de pandas: ignora (enunciado manda)
7. Variáveis dadas	usa <code>reajuste/mes</code> do enunciado, NUNCA <code>hardcode</code> do exemplo ("deve funcionar pra qualquer cenário")
8. Formato exato	copia travessão/símbolo do enunciado; <code>{x:.2f}</code> quando pedir casas; confere espaço por espaço
9. Antes de entregar	<code>Restart & Run All</code> : roda tudo de cima a baixo sem erro?
10. Varredura final	nenhuma resposta em branco · Q0 preenchida · prosa comentada com # · saídas com <code>:.2f</code> · linha em branco do modelo copiada